Structure-Aware Cross-Dialect SQL Transfer for DBMS Fuzzing Seed Construction

Abstract—DBMS fuzzing mutates initial seed test cases to explore execution behaviors and trigger crash bugs, making the quality and diversity of the initial seeds critical. Built-in test suites are commonly used for this purpose, yet many DBMSs lack them. Consequently, test cases from mature systems are often reused, but this process is hindered by SQL dialect incompatibilities. Traditional SQL translation techniques primarily operate at statement-level and are not designed for complete test-case transfer for DBMS fuzzing. Recent fuzzing-oriented approaches instead employ LLMs for end-to-end test-case transfer. However, limited knowledge of SQL structures and dialects often introduces syntactic and semantic errors, while direct transfer may simplify SQL structures and cross-statement dependencies, reducing the diversity of generated seeds.

To address these limitations, we propose Morph to construct fuzzing seeds with structure-aware cross-dialect SQL transfer. Our key insight is to decouple structural information from SQL syntax via a dialect-independent Abstract Query Template (AQT), enabling test-case-level abstraction and dialect-aware instantiation. Specifically, Morph first constructs a dialect-independent AQT from the source test case, preserving statement order and cross-statement dependencies. It then performs dialect-aware AQT instantiation to generate target-dialect SQL test cases using dialect-specific knowledge and constraints. Finally, Morph leverages parser feedback from the downstream fuzzer to repair remaining incompatibilities, ensuring that the generated test cases are parsable and effectively mutable. We evaluate Morph on four popular DBMSs: DuckDB, ClickHouse, MonetDB, and Virtuoso. Compared with the state-of-the-art seed transfer approach Sedar, Morph covers 14% more branches, finds 16 more bugs in 24 hours, and improves the overall semantic correctness of transferred seeds by 130%. More importantly, Morph uncovers 28 previously unknown DBMS bugs, including 20 confirmed by developers and 14 already fixed.

Index Terms—DBMS Fuzzing, Initial Seeds, Cross-DBMS SQL Transfer

I. Introduction

Database management systems (DBMSs) provide the essential framework for organizing and accessing data. Ensuring that a DBMS operates reliably is critical for maintaining data integrity and business continuity. However, the complex architectures and intricate implementations of modern DBMSs inevitably introduce potential bugs, which can lead to system crashes, data loss, or service interruptions [?], [?]. Timely detection of such bugs is therefore crucial for maintaining the reliability of these DBMSs.

In recent years, fuzzing has emerged as an effective technique for uncovering crashes and detecting hidden errors [?], [?], [?], [?]. Many DBMS fuzzers have already discovered lots of bugs in popular DBMSs [?], [?], [?]. These fuzzers typically generate new test cases by repeatedly mutating a set of initial seed queries to explore more execution paths. Consequently, the effectiveness of fuzzing largely depends on the quality and diversity of the initial seeds [?], [?], [?]. While established DBMSs (e.g., PostgreSQL [?]) possess extensive test suites that serve as ideal seeds, emerging engines like DuckDB [?] often lack such assets, creating a critical bottleneck that limits the depth of testing for existing fuzzers.

To construct initial seeds for these DBMSs, a common practice is to reuse test cases from other mature systems, but SQL dialect incompatibilities often hinder this process. Existing SQL translation techniques [?], [?], [?] focus on statement-level translation, which treats SQL statements independently. In contrast, DBMS fuzzing seeds are multi-statement test cases with rich cross-statement dependencies (e.g., schema references or type constraints). As a result, statement-level translation may break dependency chains, leading to execution failures when transferred across systems.

Recent fuzzing-oriented approaches instead leverage large language models (LLMs) for end-to-end test-case transfer [?], [?]. For example, Sedar [?] combines schema information and raw SQL text as input to an LLM to generate target-dialect seeds. However, such approaches still suffer from two fundamental limitations: (i) First, due to limited understanding of target SQL dialects and weak structural awareness of multi-statement programs, LLM-based translation may introduce syntactic or semantic errors during generation, leading to invalid or inconsistent query behaviors. (ii) Second, to satisfy strict grammar constraints imposed by downstream fuzzers, generated test cases are often sanitized by removing or commenting out unsupported constructs, which may simplify complex SQL structures and weaken execution behaviors.

To address the limitations of existing approaches, we propose a structure-aware method for cross-dialect transfer of SQL test cases. Our key insight is to decouple structural information from SQL syntax via a dialect-independent Abstract Query Template (AQT), enabling test-case-level abstraction and dialect-aware instantiation. We observe that, although different SQL dialects vary in function naming or syntactic sugar, they typically share an invariant abstract query logic that governs the underlying operations and relationships between data elements. By separating the core query logic from dialect-specific syntax, test cases can be transferred across DBMSs while preserving structurally rich execution behaviors that are essential for fuzzing. This enables fuzzing to be more accurate and comprehensive, as it retains the essential behaviors of the test cases while avoiding transfer failures caused by superficial syntactic differences.

However, realizing this idea has three challenges: (1) First, SQL logic is often tightly coupled with dialect-specific syntax, requiring the separation of syntactic differences while preserving the original structure, dependencies, and semantic richness. (2) Second, ensuring target-side semantic correctness during SQL transfer is non-trivial, as operators, functions, and type conversions may differ across dialects, requiring carefully constrained transfer. (3) Third, transferred test cases must satisfy strict parsing and mutation requirements imposed by downstream fuzzers, while preserving their structural richness under these constraints.

To address these challenges, we propose Morph, a structure-aware cross-dialect DBMS fuzzing framework for constructing high-quality initial seeds and enabling effective fuzzing. Morph consists of three steps: (1) First, it parses each SQL test case into an Abstract Query Template (AQT), capturing statement order, cross-statement dependencies, and clause-level structure while abstracting dialect-specific syntax. (2) Second, Morph instantiates AQT nodes into target-dialect constructs under target-dialect constraints while propagating the evolving target schema context across statements, with localized rule generation and repair when necessary. LLMs are used for bounded rule generation or local repair under structural constraints. (3) Finally, it refines generated test cases to satisfy fuzzing requirements using runtime feedback, converting unsupported constructs into parsable forms while preserving their execution behavior as much as possible.

We implement Morph and evaluate it on four popular DBMSs, namely DuckDB, ClickHouse, MonetDB, and Virtuoso. Although these DBMSs have been extensively tested, Morph still uncovers 28 previously unknown bugs, including 20 confirmed and 14 already fixed. These bugs have received attention from developers. For example, all reported bugs of MonetDB have been incorporated into its official NextRelease milestone [?]. We further compare Morph with state-of-the-art DBMS fuzzing tools, including SQLancer, Squirrel, Griffin, and Sedar. Across the four DBMSs under 24 hours, Morph achieves 14%–528% higher branch coverage and discovers 16–22 more bugs than others. To evaluate seed transfer quality, we also compare Morph against representative relevant techniques, including SQLGlot, jOOQ, CrackSQL, RISE, and Sedar. Morph outperforms these baselines by 5%–62% in syntax correctness, 100%–450% in semantic correctness, and 11%–253% in preservation of SQL functions. Finally, we conduct an ablation study to show the contribution of each component in Morph. In summary, the paper makes the following contributions:

1) We find that many DBMSs lack high-quality initial seeds for efficient fuzzing. LLM-based end-to-end translation often introduces errors and structural simplifications, and limited SQL grammar support further hinders mutation and degrades query semantics, reducing testing effectiveness.
2) We propose Morph, a dependency-preserving whole-test-case seed transfer framework. Instead of translating isolated SQL statements, Morph encodes each source test case as an AQT, instantiates it under target-dialect constraints, and adapts remaining incompatibilities into fuzzer-parsable forms while preserving testing-relevant structures.
3) We implement Morph and evaluate it on four DBMSs, where it discovers 28 previously unknown bugs (20 confirmed, 14 fixed). Compared to the state-of-the-art method Sedar, Morph improves the overall semantic correctness of transferred seeds by 130%, achieving a 14% increase in branch coverage.

II. Background and Motivation

SQL Dialects. Database Management Systems (DBMSs) underpin modern data-driven applications and expose high-level declarative query languages for data interaction. Although SQL is standardized, real-world DBMSs implement diverse SQL dialects with substantial syntactic and semantic variations. For example, PostgreSQL uses AGE(timestamp) for date arithmetic, whereas DuckDB requires date_diff(). Similarly, type casting syntax varies between ::INTERVAL in PostgreSQL and CAST(... AS INTERVAL) in standard SQL. These dialect gaps pose substantial barriers for reusing test assets across different DBMSs.

DBMS Fuzzing. DBMS Fuzzing has emerged as a core technique for discovering database system crashes [?], [?], [?]. By continuously mutating existing SQL test cases to traverse diverse execution paths, DBMS fuzzers can effectively expose corner cases and potential bugs. The effectiveness of fuzzing largely depends on the quality and diversity of initial seed test cases. Due to the diverse features of DBMSs, fuzzers lacking high-quality initial seeds with specific syntactic features (e.g., recursive CTEs or window functions) are often confined to shallow parsing logic, leaving deep architectural bugs undetected. However, while mature DBMSs such as PostgreSQL provide rich test suites, many emerging systems lack such resources, creating a bottleneck for effective fuzzing.

Cross-Dialect Seed Transfer. To address seed scarcity for emerging DBMS testing, cross-DBMS SQL transfer has emerged as an active research direction [?], but this is challenging due to SQL dialect differences. As shown in Table ??, most existing tools do not explicitly preserve cross-statement dependencies, making them unsuitable for test-case-level transfer in fuzzing. Specifically, existing

SQL translation techniques [?], [?], [?] focus on statement-level translation, which treats each SQL statement independently. In contrast, fuzzing seeds are multi-statement test cases that include complex interactions among DDL, DML, and DQL statements. Recent work such as Sedar [?] explores fuzzing-oriented seed transfer using LLMs. While it improves automation by translating complete test cases, it still does not explicitly preserve cross-statement dependencies during migration.

Motivating Example. We next present a motivating example to illustrate the limitations of LLM-based fuzzing-oriented seed transfer methods such as Sedar. LLM-based cross-dialect transfer methods help in the generation of initial fuzzer seeds but may introduce syntax or semantic errors due to hallucinations. Moreover, many fuzzers cover only a subset of SQL grammar. Existing methods typically comment out unsupported features to support seed mutation after transfer, which alters the query and causes structural simplification.

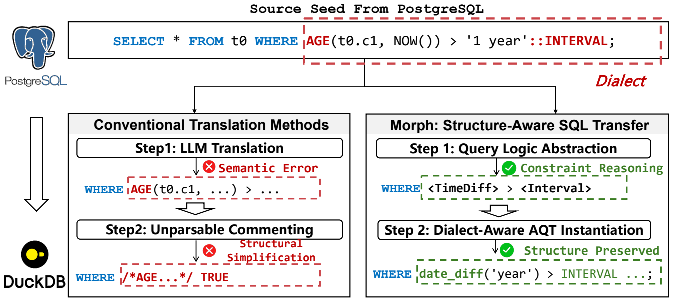

Source Seed From PostgreSQL
SELECT * FROM t0 WHERE AGE(t0.c1, NOW()) > '1 year'::INTERVAL;
PostgreSQL
Dialect
DuckDB
Conventional Translation Methods
Step1: LLM Translation
Semantic Error
WHERE AGE(t0.c1, ...) > ...
Step2: Unparsable Commenting
Structural Simplification
WHERE /*AGE...*/ TRUE
Morph: Structure-Aware SQL Transfer
Step 1: Query Logic Abstraction
Constraint Reasoning
WHERE <TimeDiff> > <Interval>
Step 2: Dialect-Aware AQT Instantiation
Structure Preserved
WHERE date_diff('year') > INTERVAL ...;

Fig. 1. Conventional end-to-end translation methods like Sedar suffer from semantic error and structural simplification (Left). Morph decouples intent via AQT, successfully instantiating the equivalent date_diff logic (Right).

Limitations of Existing Approaches. Figure ?? illustrates the semantic deviation and logic loss Sedar experiences when transferring a test case from PostgreSQL to DuckDB, which contains dialect-specific features like the AGE function in the WHERE clause and the ::INTERVAL type cast. With limited knowledge of DuckDB's syntax, the LLM hallucinates and preserves AGE, causing syntax and semantic errors. To satisfy parsing constraints, Sedar comments out AGE and replaces the WHERE condition with True, producing a syntactically valid but simplified query that loses the original verification intent and fails to exercise deep internal states. As a result, the migrated seed has limited utility for subsequent effective mutation in DBMS fuzzing.

TABLE I

Capability comparison of existing SQL translation systems for test-case-level DBMS fuzzing seed transfer.

Tool	DDL	DML	DQL	Test Case Migration	Cross-stmt. Deps.
SQLGlot [?]	✓	✓	✓	×	×
jOOQ [?]	✓	✓	✓	×	×
RISE [?]	△	△	✓	×	×
CrackSQL [?]	△	✓	✓	×	×
QTRAN [?]	△	△	✓	×	×
Sedar [?]	△	△	✓	✓	×
Morph	✓	✓	✓	✓	✓

Cross-stmt. Deps. denotes explicit preservation of schema, data, and state dependencies across statements. ✓: explicit support; △: limited support; ×: not supported.

Basic Idea of Morph. The core idea of Morph is to thoroughly decouple logical intent from syntactic representation, shifting from unstable textual translation to more robust structural recursive substitution. As shown in Figure ??, Morph does not perform erroneous syntactic translation, but instead abstracts the dialect-specific AGE function into a generic <TimeDiff> intent. It then performs a high-fidelity instantiation into DuckDB's equivalent syntax. By ensuring the type casting and window function partition logic are preserved, Morph generates a high-fidelity seed that retains the original filtering complexity. By maintaining these complex semantics during transfer, Morph ensures the generated seeds are effective for validation, enabling them to reach deeper code paths within the DBMS and uncover intricate logic bugs that might be unreachable by Sedar.

III. Approach

Morph is a structure-aware cross-dialect DBMS fuzzing framework for generating effective seeds for the target DBMSs. As Figure ?? shows, it first abstracts each source test case into an AQT that captures statement templates, cross-statement dependencies, and schema context, then recursively instantiates abstract nodes under target-dialect constraints using retrieved or generated rules. When generated outputs are rejected by the target parser or executor, Morph performs feedback-driven local repair instead of discarding the structure, producing fuzzer-compatible seeds while preserving testing-relevant features.

A. Structure-Preserving Abstraction

This step decouples the logical intent of a test case from its dialect-specific syntax. The challenge is that dialect-specific constructs, such as proprietary functions, type casts, and interval operators, are often embedded within hierarchical query structures (e.g., nested subqueries and CTEs). As a result, extracting reusable logic requires preserving relational structure, scoping boundaries, and cross-statement dependencies, rather than relying on surface-level text replacement. We address this by formalizing each seed as an AQT that records ordered statement templates, dependency relations, and cumulative schema context as a structured object amenable to manipulation. We begin with the following definitions.

Formal Definition. To support syntax-aware query abstraction and decouple the logical intent of a test case, we

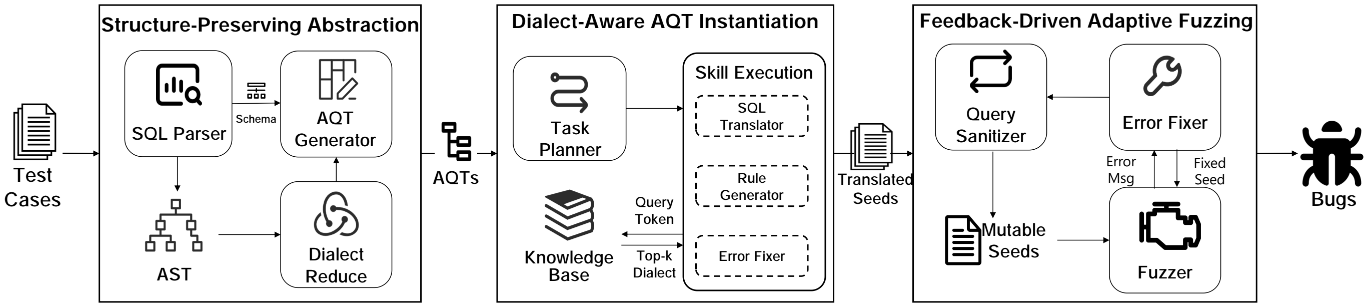


Fig. 2. Overview of the Morph. Morph generates high-quality cross-dialect test cases following three steps: (1) Structure-Preserving Abstraction. Morph first parses SQL into an AST and extracts schema information, then replaces dialect-specific syntax with generic labels while preserving cross-statement dependencies to construct AQTs. (2) Dialect-Aware AQT Instantiation. Morph uses a task planner to decompose test case transformations and instantiates AQT nodes by retrieving dialect evidence from a hybrid knowledge base, invoking the LLM only for bounded rule generation and local error correction. (3) Feedback-Driven Adaptive Fuzzing. It fixes syntax incompatibilities, then uses DBMS execution feedback to repair remaining errors, and finally marks mutable regions in the mutable seeds to guide fuzzer mutations.

introduce the SQL abstraction rules and AQT. They are defined as follows:

Definition 1 (SQL Abstraction Rule): An abstraction rule maps a dialect-specific SQL construct to an abstract semantic label:

$$[\text{Function} \mid \text{Type} \mid \cdots] \rightarrow [\langle\text{FuncTag}\rangle \mid \langle\text{TypeCast}\rangle \mid \cdots].$$

where Source is a dialect-specific SQL element and Target is its corresponding abstract label that represents the semantic intent. For example, a rule for interval computation can be represented as $\text{AGE}(\cdot) \rightarrow \langle\text{TimeDiff}\rangle$.

Definition 2 (Abstract Query Template): Given a SQL test case $\mathcal{C}$, its AQT is a five-tuple:

$$\mathcal{T} = \langle \mathbf{U}, \Delta, \mathcal{D}, \mathcal{G}_{tc}, \mathcal{H} \rangle.$$

Here, $\mathbf{U} = \langle \mathcal{U}_1, \ldots, \mathcal{U}_n \rangle$ is the ordered sequence of statement templates, where each $\mathcal{U}_i = \langle kind_i, S^i_{norm}, \mathcal{G}^i, R_i, W_i \rangle$ records the statement type, normalized skeleton, statement-level intra-statement graph, read set, and write set of statement s_i. $\Delta \subseteq \{(i,j) \mid i < j\}$ captures cross-statement dependencies, where $(i,j) \in \Delta$ iff $R_j \cap W_i \neq \emptyset$. $\mathcal{D}$ records the cumulative schema context after executing $\langle s_1, \ldots, s_n \rangle$, including table definitions, column types, constraints, and object-state summaries that are propagated as source-side constraints during target-side instantiation. $\mathcal{G}_{tc} = (V_{tc}, E_{tc})$ is the test-case dependency graph, where $V_{tc} = \{1, \ldots, n\}$ and $E_{tc} = \Delta$; thus, Δ is the dependency set and $\mathcal{G}_{tc}$ is its graph representation for dependency-preserving ordering. $\mathcal{H}$ is a structural hash over the ordered normalized skeletons and dependency edges for deduplication and cache indexing.

Structure-Aware AQT Generation. Unlike prior statement-level translators that process each SQL statement in isolation and emit a statement-local rewrite rule, Morph treats the whole test case as the abstraction unit. Algorithm ?? summarizes this extraction process. It first initializes the statement-unit list, dependency set, and extraction context. For each statement, ExtractStmtTemplate abstracts dialect-specific syntax into a normalized template while preserving scoped structures such as CTEs and subqueries (Line ??). Morph then derives read/write sets, records cross-statement dependencies, appends the template, and updates the context for subsequent statements (Lines ??–??). After the loop, Morph snapshots the final context, builds the dependency graph, computes the structural hash, and returns the AQT five-tuple (Lines ??–??). This representation enables SQL test cases to be manipulated as structured graph-based objects rather than flat strings.

Algorithm 1: AQT Extraction

```
Input : Test case $\mathcal{C} = \langle s_1, \ldots, s_n \rangle$; initial context $\mathcal{S}_0$;
        rules $\mathcal{R}$
Output: AQT $\mathcal{T} = \langle \mathbf{U}, \Delta, \mathcal{D}, \mathcal{G}_{tc}, \mathcal{H} \rangle$
1  $\mathbf{U} \leftarrow [\,]$; $\Delta \leftarrow \emptyset$; $Ctx \leftarrow \mathcal{S}_0$;
2  foreach $s_i$ in $\mathcal{C}$ do
3      $\mathcal{U}_i \leftarrow$ ExtractStmtTemplate($s_i, Ctx, \mathcal{R}$) ;
4      $(R_i, W_i) \leftarrow$ AccessSets($\mathcal{U}_i$) ;
5      $\Delta \leftarrow \Delta \cup \{(j,i) \mid j < i \wedge R_i \cap W_j \neq \emptyset\}$;
6      $\mathbf{U}$.Append($\mathcal{U}_i$);
7      $Ctx \leftarrow$ UpdateContext($Ctx, s_i$) ;                // Evolve
         schema/state
8  $\mathcal{D} \leftarrow Ctx$ ;
9  $\mathcal{G}_{tc} \leftarrow (\{1, \ldots, |\mathbf{U}|\}, \Delta)$;
10 $\mathcal{H} \leftarrow$ SHA256($\bigoplus_i$ Skeleton($\mathcal{U}_i$) $\oplus \Delta$);
11 return $\langle \mathbf{U}, \Delta, \mathcal{D}, \mathcal{G}_{tc}, \mathcal{H} \rangle$ ;
```

For example, Figure ?? shows one statement from a source SQL test case containing a nested subquery, where the outer query filters records using an interval computation, e.g., “SELECT ... WHERE u.created_at >(SELECT AGE(...) FROM ...)”. The statement is first parsed into an AST, and structural boundaries are identified during recursive traversal. The SUBQUERY root node is recognized as a boundary, decomposing the query into an outer block (ID 1) and an inner subquery (ID 2). The abstraction engine then applies rules $\mathcal{R}$ to normalize dialect-specific constructs, mapping AGE(...), ::INT, and MIN(.) into <TimeDiff>, <TypeCast>, and <AggMin>. Finally, a statement template is constructed, capturing the parent-child relationship via a pointer <Ptr:2> and producing a normalized skeleton S^i_{norm} that preserves logic while abstracting dialect differences.

B. Dialect-Aware AQT Instantiation

This step instantiates AQTs into syntactically valid target seed sequences while preserving semantic-rich structures from the source seed. The instantiation respects the dependency order in $\mathcal{G}_{tc}$: DDL and DML statements are instantiated before queries that depend on them, and the evolving target context is propagated across statements to maintain cross-statement type, reference, and state consistency. The challenge is preserving target-side semantic correctness during recursive AQT-node instantiation without breaking the original test-case order. To address it, Morph transforms the instantiation task into a closed-loop reasoning process built on a Retrieval-Augmented Generation (RAG) architecture, which grounds LLM outputs in retrieved dialect-specific evidence to reduce hallucinations.

We formulate dialect translation as a two-level instantiation problem in which the outer level walks the statement dependency graph while preserving the original sequence for independent statements, and the inner level recursively instantiates each statement-level graph $\mathcal{G}^i$ in a bottom-up manner. To guarantee that the generated queries remain semantically consistent with the target dialect during test-case instantiation, the system employs a Dialect-Aware AQT Instantiation process as formalized in Algorithm ??. At the outer level, the algorithm first computes a dependency-preserving statement order over $\mathcal{G}_{tc}$ while keeping the original order for independent statements (Line ??). For each statement, it invokes InstantiateStmt with the source constraints and current target context, appends the generated query, and updates the target context for subsequent statements (Lines ??–??). After all statements are instantiated, the generated target sequence is indexed by $(\mathcal{H}, D_t)$ for future reuse. At the inner level, InstantiateStmt recursively instantiates the statement-level graph using retrieved or generated local mappings and target-parser feedback to keep fragments executable. Finally, the instantiated statements form the target seed sequence $\mathcal{Q}_{target}$.

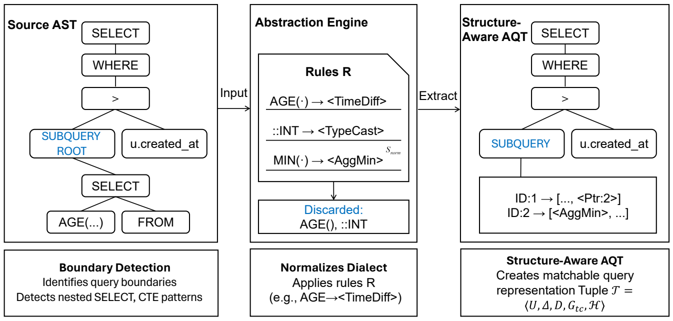


Fig. 3. An example of Structure-Preserving Abstraction. It first detects and normalizes dialect-specific SQL features to abstract labels using abstraction rules, and produces structure-aware AQTs.

Algorithm 2: AQT Instantiation

```
Input  : AQT $\mathcal{T} = \langle \mathbf{U}, \Delta, \mathcal{D}, \mathcal{G}_{tc}, \mathcal{H} \rangle$; target dialect $D_t$;
         Hybrid KB $HKB$; threshold $\delta$
Output : Compatible seed sequence $\mathcal{Q}_{target}$
$StmtOrder \leftarrow$ StableTopologicalSort($\mathcal{G}_{tc}, \langle 1, \ldots, |\mathbf{U}| \rangle$) ;
  // Respect deps; keep original order
$\mathcal{Q}_{target} \leftarrow [\,]$; $C_s \leftarrow \mathcal{D}$; $C_t \leftarrow \emptyset$;
foreach $i$ in $StmtOrder$ do
    $Q_i \leftarrow$ InstantiateStmt($\mathcal{U}_i, D_t, HKB, \delta, C_s, C_t$) ;
    $\mathcal{Q}_{target}$.Append($Q_i$);
    $C_t \leftarrow$ UpdateContext($C_t, Q_i$) ;          // Propagate
      context
IndexArtifact(($\mathcal{H}, D_t$), $\mathcal{Q}_{target}$) ;   // Cache key outside
  recursion
return $\mathcal{Q}_{target}$;
```

Test-Oriented Skill Execution. The above instantiation procedure is realized through Test-Oriented Skill Execution, a method that coordinates three specialized skills under the target-dialect constraints.

SQL Translator: This skill instantiates a token into a dialect-compliant fragment by integrating the input segment with function signatures from the Vector Index. For example, it maps <TimeDiff> to DuckDB's date_diff signature.

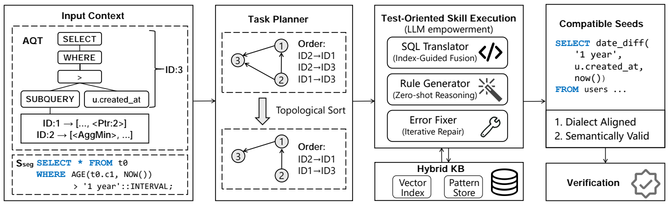
Rule Generator: When retrieval confidence for a feature falls below a threshold, this skill synthesizes a candidate local rule via the LLM's zero-shot reasoning over the AST context and few-shot examples. The candidate is cached in the Hybrid KB only after it passes target parser validation.

Error Fixer: When the target parser rejects a fragment, this skill localizes the error using both the compiler feedback and the AST context, then applies structure-preserving rewrites via constrained few-shot examples. This avoids degenerate repairs such as clause removal, keeping the output a Compatible Seed with its original complexity.

The three skills are grounded in a Hybrid KB with two complementary stores: the Abstract Pattern Store keeps deduplicated structural skeletons (e.g., JOIN/WINDOW templates) to preserve query topology, while the Vector Index retrieves top-k dialect evidence (function signatures and operator semantics) for each token τ, grounding generation in concrete evidence rather than free-form synthesis.

As shown in Figure ??, since ID1 depends on the nested segment ID2 (Order=[`ID2` $\rightarrow$ `ID1`]), Morph first instantiates subquery ID2 by mapping <AggMin> to MIN(..) and <TypeCast> to a dialect-valid CAST(.. AS ..), and validates it via parser feedback. It then substitutes the validated fragment into the outer predicate of ID1, mapping <TimeDiff> to DuckDB's date_diff, yielding a

Compatible Seed that preserves the nested time-window logic.


Input Context
AQT
SELECT
WHERE
>
SUBQUERY
u.created_at
ID:3
ID:1 → [..., <Ptr:2>]
ID:2 → [<AggMin>, ...]
Task Planner
Order:
ID2→ID1
ID2→ID3
ID1→ID3
Topological Sort
Order:
ID2→ID1
ID1→ID3
Test-Oriented Skill Execution
(LLM empowerment)
SQL Translator
(Index-Guided Fusion)
Rule Generator
(Zero-shot Reasoning)
Error Fixer
(Iterative Repair)
Hybrid KB
Vector Index
Pattern Store
Compatible Seeds
1. Dialect Aligned
2. Semantically Valid
Verification

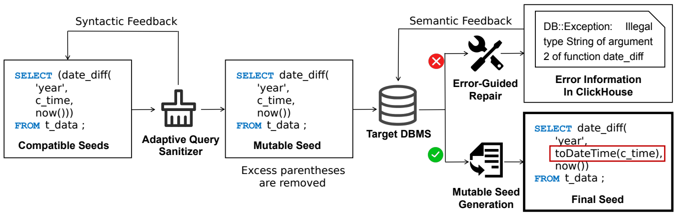
```
Sseg SELECT * FROM t0
     WHERE AGE(t0.c1, NOW())
           > '1 year'::INTERVAL;
```

```
SELECT date_diff(
    '1 year',
    u.created_at,
    now())
FROM users ...
```

Fig. 4. An example of the RAG-enhanced Dialect-Aware AQT Instantiation.

C. Feedback-Driven Adaptive Fuzzing

Some DBMS features exceed the grammar supported by existing fuzzers, making otherwise executable target seeds difficult to mutate. The challenge is to adapt such features into fuzzer-compatible forms while preserving syntax correctness, semantic consistency, and mutation opportunities. Morph addresses this challenge using parsing and execution feedback to repair unsupported constructs through a structure-preserving fallback process. As shown in Figure ??, compatible seeds are sanitized, executed in the target DBMS, repaired when necessary, and annotated with mutable regions for fuzzing.


Syntactic Feedback
Compatible Seeds
Adaptive Query Sanitizer
Mutable Seed
Excess parentheses are removed
Target DBMS
Semantic Feedback
Error-Guided Repair
Error Information In ClickHouse
Mutable Seed Generation
Final Seed

```
SELECT (date_diff(
    'year',
    c_time,
    now()))
FROM t_data ;
```

```
SELECT date_diff(
    'year',
    c_time,
    now())
FROM t_data ;
```

```
DB::Exception: Illegal type String of argument 2 of function date_diff
```

```
SELECT date_diff(
    'year',
    toDateTime(c_time),
    now())
FROM t_data ;
```

Fig. 5. Collaborative Workflow of Adaptive Query Sanitizing, Error-Guided Repairing, and Mutable Seed Generation in Feedback-Driven Adaptive Fuzzing.

Specifically, when a transferred seed Q_{target} still violates the fuzzer grammar, the sanitizer localizes the unsupported construct to its AQT node and repairs it using parser feedback rather than discarding it. For example, it removes excess brackets around date_diff in Figure ??, producing a syntactically valid Mutable Seed.

When an error signal is received (e.g., "Syntax error near SELECT"), Morph applies four fallbacks in order: existing sanitization mappings, RAG-retrieved replacements, source-side execution followed by constant substitution, and finally minimal commenting. Thus, destructive commenting is used only after structure-preserving repairs fail. Once a seed passes the fuzzer parser, Morph marks data-dependent literals and predicates as mutable regions. This guides mutation toward logic-altering changes while keeping the repaired syntactic skeleton stable.

IV. Evaluation

Morph is implemented in Python and integrates Griffin [?], a coverage-guided mutation fuzzer, as the downstream execution engine. For the LLM used in Test-Oriented Skill Execution and rule generation, we employ Gemini 2.5-Flash-Lite [?] as the primary language model due to its strong code understanding capabilities and cost efficiency. We set the temperature parameter to 0.2 to balance determinism and output diversity, as justified by the sensitivity analysis discussion in Section ??. We evaluate Morph to assess its performance in cross-dialect seed transformation and its practical impact on DBMS fuzzing effectiveness. Specifically, our evaluation focuses on the following research questions:

- RQ1: Can Morph find unknown bugs in DBMSs?
- RQ2: How does Morph compare to other fuzzing tools?
- RQ3: How does Morph compare to other translation methods in terms of seed validity and semantic richness?
- RQ4: What is the contribution of each component in Morph to the overall performance?

A. Experimental Setup

Target DBMSs and Environment. To evaluate the generalization of Morph across diverse architectures, we selected four widely used DBMSs according to the DB-Engines Ranking [?]: MonetDB [?] v11.51, Virtuoso [?] v7.2, DuckDB [?] v1.4.3, and ClickHouse [?] v24.8 LTS. These systems cover diverse OLAP and multi-model architectures with distinct SQL dialects. We did not include traditional row-store OLTP systems, such as MySQL and PostgreSQL, as target DBMSs because they are used as source DBMSs for seed collection. Instead, we prioritized OLAP and multi-model systems to more rigorously evaluate Morph's ability to bridge substantial dialect and architectural gaps, such as transferring seeds from row-store systems like MySQL to column-store systems like DuckDB. All experiments were conducted on a Linux server equipped with an Intel Xeon CPU and 256 GB of RAM. To ensure reproducibility and isolation, each DBMS instance ran within a separate Docker container.

Collected Test Cases. Following the prior methodology [?], we collected built-in test cases from mature DBMS repositories (i.e., SQLite [?], MySQL [?], and PostgreSQL [?]) for transfer. The dataset aggregates 447,357 SQL statements across 6,875 files, totaling 55.7 MiB of query text.

Evaluation Metrics. We employ three metrics to evaluate Morph. First, to assess transfer quality, we measure Syntactic Correctness, defined as the proportion of generated seeds that pass the target parser, and Semantic Correctness, defined as the proportion of generated seeds that execute without runtime errors. Here, semantic correctness is defined by executability in the target DBMS without runtime errors, not by result equivalence between the source and target DBMSs. We separately measure semantic richness by counting valid functions, data types, and keywords preserved in executable transferred seeds. Second, to quantify the effectiveness in improving fuzzing, we use LLVM's SanitizerCoverage to track Branch Coverage over 24-hour campaigns. We report the average results across five independent runs to mitigate randomness. Third, for real-world bug discovery, we report the number of detected unique bugs.

B. RQ1: Real-World Bug Detection

RQ1 evaluates Morph's real-world bug-finding impact over the full campaign. We deployed Morph on the development versions of the four target DBMSs. As summarized in Table ??, the campaign uncovered 28 unique bugs. Among these issues, 20 have been confirmed by the vendors, of which 14 critical bugs have already been fixed.

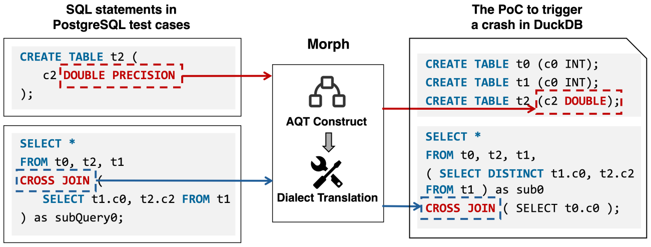
These bugs have received positive feedback and careful attention from the corresponding database developers. For instance, regarding the fixed bug in DuckDB (#205xx), the developers appreciated the report exposing an intricate type inference flaw in the Executor, which was triggered by preserving complex cross-table correlations from PostgreSQL. They merged a fix within days, further validating that Morph successfully identified deep-seated bugs missed by existing test suites. Moreover, all reported bugs of MonetDB have been incorporated into its official NextRelease milestone [?]. The detected bugs are not limited to the surface level (e.g., Parser) but penetrate deep into the DBMS kernels. As shown in the second column of Table ??, the bugs span critical subsystems including the Optimizer, Binder, and Storage Engine.

Case Study: An internal error in DuckDB. To illustrate the effectiveness of Morph, we examine a fixed bug in DuckDB (#205xx) triggered by a complex query involving a nested CROSS JOIN and specific DDL configurations. Generally, in SQL dialects, join operations and subqueries are standard; however, the structural interaction between specific join types and aliased subqueries can induce a crash bug in DuckDB when executing the statement shown in the PoC. In this case, the source logic was derived from a PostgreSQL test case featuring a CREATE TABLE statement with DOUBLE PRECISION and a query containing a nested CROSS JOIN.

TABLE II
Statistics of new DBMS bugs detected by Morph.

Confirmed/Detected	Component	Bug Type
MonetDB (15/17)	optimizer	SEGV(6), NPD(1), AF(1)
	server	SEGV(4), DBZ(1), AF(2)
	function	SEGV(1), NPD(1)
Virtuoso (2/5)	server	SEGV(4), UAF(1)
DuckDB (2/4)	executor	AF(1), DBZ(1)
	binder	SEGV(1)
	function	OOM(1)
ClickHouse (1/2)	function	SEGV(1)
	server	AF(1)
Total	9 components	20 confirmed, 28 detected

UAF: Use-After-Free; AF: Assertion Failure; NPD: Null Pointer Dereference; DBZ: Divide-by-Zero; SEGV: Segfault; OOM: Out of Memory.


SQL statements in PostgreSQL test cases
Morph
AQT Construct
Dialect Translation
The PoC to trigger a crash in DuckDB

```
CREATE TABLE t2 (
    c2 DOUBLE PRECISION
);
```

```
SELECT *
FROM t0, t2, t1
CROSS JOIN (
    SELECT t1.c0, t2.c2 FROM t1
) as subQuery0;
```

```
CREATE TABLE t0 (c0 INT);
CREATE TABLE t1 (c0 INT);
CREATE TABLE t2 (c2 DOUBLE);
```

```
SELECT *
FROM t0, t2, t1,
( SELECT DISTINCT t1.c0, t2.c2
FROM t1 ) as sub0
CROSS JOIN ( SELECT t0.c0 );
```

Fig. 6. An internal error in DuckDB (Bug #205xx). Morph translates a test case of PostgreSQL to DuckDB while preserving data types and join logic.

Morph's effectiveness stems from its structure-aware transfer. Take the case as an example, in PostgreSQL, nested joins involving a table with DOUBLE PRECISION and a subquery are well tested, whereas DuckDB lacks sufficient coverage for such combinations. Conventional approaches often simplify schemas (e.g., converting DOUBLE to INT) or flatten nested queries to ensure compatibility, sacrificing semantic fidelity. In contrast, Morph preserves the full structural complexity of the original seed, forcing DuckDB to perform precise type inference and vector allocation for correlated joins and thereby exercising execution paths that simplified queries would not reach. This high-fidelity transfer exposes a critical flaw in the executor logic that remains undetected when structural complexity is removed.

> Answer to RQ1: Morph successfully exposed 28 unknown bugs in four popular DBMSs, affecting core components including the Optimizer and Storage Engine. Among them, 20 bugs have been confirmed, and 14 bugs have already been fixed by DBMS vendors.

C. RQ2: Comparison with Other Fuzzing Tools

RQ2 evaluates Morph's effectiveness by comparing it to other fuzzing tools under the same setting. We compared Morph against four state-of-the-art DBMS testing tools in 24 hours: SQLancer [?] (logic/crash baseline [?], [?], [?]), Squirrel [?] (default built-in seed corpus [?]), Griffin [?] (grammar-free mutation baseline), and Sedar [?]. After fuzzing, we normalized coverage via a unified dry run across all configurations, and deduplicated crashes using stack-trace hashing followed by manual inspection for verification.

As illustrated in Table ??, the transferred seeds lead to the highest branch coverage across all evaluated DBMSs, covering 422,130 branches in total. This corresponds to overall improvements of 528%, 60%, 54%, and 14% over

TABLE III
Number of branches covered by different tools (24h).

DBMS	SQLancer	Squirrel	Griffin	Sedar	Morph
MonetDB	-	43,747	61,981	76,142	81,486
Virtuoso	-	26,302	28,017	36,583	40,183
DuckDB	22,188	96,434	96,512	114,876	144,822
ClickHouse	25,618	-	87,388	141,215	155,639
Total	47,806	166,483	273,898	368,816	422,130
Improvement*	528%↑	60%↑	54%↑	14%↑	-

* Calculated only for commonly supported DBMSs.

SQLancer, Squirrel, Griffin, and Sedar, respectively, on their supported targets. Figure ?? further shows earlier coverage growth and continued branch gains after 12 hours, while several baselines plateau sooner.

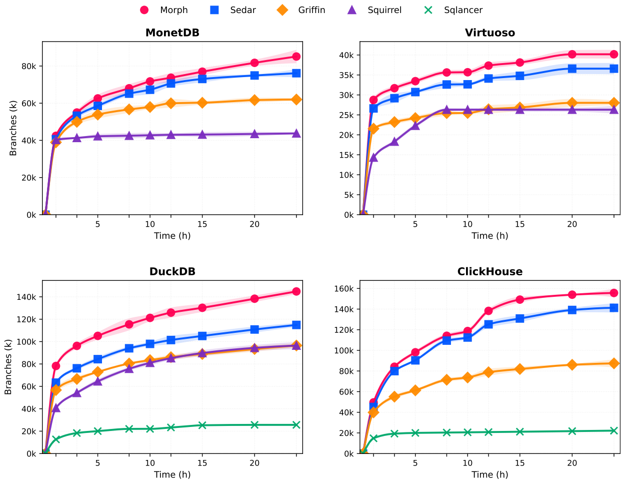


Fig. 7. Coverage growth over 24-hour fuzzing campaigns.

The coverage improvement also brings more detected bugs. As shown in Table ??, the campaign exposes significantly more unique bugs than the baselines, including 16 more bugs than Sedar across the four DBMSs. This proves that coverage gains are not merely statistical but functionally impactful, enabling the detection of bugs that remain unreachable to baselines. Unlike Sedar, which sacrifices semantic depth through destructive sanitization, Morph employs structure-aware transfer to retain the full logical complexity of source test cases. By preserving executable semantics during transfer, Morph enables fuzzers to bypass early engine rejections, exercise deeper code logic, and trigger more bugs.

TABLE IV
Number of bugs detected by different tools (24h).

DBMS	SQLancer	Squirrel	Griffin	Sedar	Morph
MonetDB	-	0	0	3	14
Virtuoso	-	0	0	2	3
DuckDB	0	0	0	1	3
ClickHouse	0	-	0	0	2
Total	0	0	0	6	22
Increment	22↑	22↑	22↑	16↑	-

> Answer to RQ2: Morph covers 528%, 60%, 54%, and 14% more branches than SQLancer, Squirrel, Griffin, and Sedar, respectively, and discovers 22, 22, 22, and 16 more bugs than SQLancer, Squirrel, Griffin, and Sedar, respectively. The results demonstrate the effectiveness of Morph in enhancing DBMS fuzzing.

D. RQ3: Effectiveness of Transfer of Morph

For translation effectiveness, we compare Morph against six baselines, including No translation, SQLGlot [?], jOOQ [?], CrackSQL [?], RISE [?], and Sedar [?]. The No translation strategy directly executes the original test cases collected from source DBMSs without modification. SQLGlot and jOOQ represent off-the-shelf SQL dialect translators, while CrackSQL and RISE represent recent SQL translation systems. Sedar represents the current state-of-the-art LLM-based transfer tool in constructing initial seeds in fuzzing. We reproduced Sedar's complete pipeline, including schema capture, end-to-end LLM translation, and its comment-out sanitization strategy.

We evaluate these methods from two aspects: transferred-seed validity and SQL-feature richness. This evaluation targets fuzzing seed quality rather than cross-DBMS result equivalence: valid seeds must execute on the target DBMS, and rich seeds should retain diverse SQL features for mutation.

Validity of the Transferred Test Cases. As shown in Table ??, Morph outperforms the baseline methods by 5%–62% in syntax correctness and 100%–450% in semantic correctness. Directly executing source seeds yields low semantic correctness, dropping to 0.097 on Virtuoso and reaching only 0.284 even in the best case. Existing translators improve syntactic acceptance but still leave a semantic gap; on DuckDB, SQLGlot, CrackSQL, and RISE reach at least 0.738 syntactic correctness, yet no more than 0.314 semantic correctness.

Existing SQL translation tools primarily operate at the statement level and are designed for query migration rather than fuzzing seed transfer. For example, RISE's statement-level design is particularly limited in seed transfer scenarios. In the SQLite-to-DuckDB setting, our test-case-level transfer improves semantic correctness from 0.314 to 0.776. When the translated outputs are used as initial seeds for fuzzing, Morph further achieves 34% higher branch coverage than RISE and detects three bugs in DuckDB, whereas RISE triggers none.

Compared with Sedar, Morph performs better in each DBMS, with 5%–19% higher syntactic correctness and 100%–272% higher semantic correctness across the tested DBMSs. As discussed in Section ?? (Table ??), these low semantic scores stem from the lack of explicit cross-statement dependency preservation in statement-level translators such as SQLGlot, jOOQ, CrackSQL, and RISE, or from the lack of explicit schema-state propagation in Sedar, which can break type or reference consistency when DDL-created schema state is not propagated to subsequent statements. These gains confirm that Dialect-Aware AQT Instantiation maps source intent into target-dialect constraints while preserving parsability.

TABLE V
Syntactic and semantic correctness of transferred seeds.

Method	MonetDB		Virtuoso		DuckDB		ClickHouse	
	Syn.	Sem.	Syn.	Sem.	Syn.	Sem.	Syn.	Sem.
No translation	0.589	0.236	0.324	0.097	0.674	0.284	0.613	0.182
SQLGlot	-	-	-	-	0.738	0.186	0.665	0.193
jOOQ	-	-	-	-	0.652	0.136	0.554	0.175
CrackSQL	-	-	-	-	0.740	0.211	0.717	0.264
RISE	0.830	0.266	0.545	0.178	0.857	0.314	0.764	0.278
Sedar	0.781	0.388	0.562	0.169	0.895	0.365	0.821	0.315
Morph	0.929	0.776	0.670	0.629	0.939	0.748	0.895	0.694
vs. Sedar	+19%	+100%	+19%	+272%	+5%	+105%	+9%	+120%

Syn./Sem.: syntactic/semantic correctness; “-”: unsupported dialects.

Semantic Richness of the Transferred Test Cases. Beyond validity, we measure whether transferred seeds retain testing value by counting valid functions, data types, and keywords. As shown in Table ??, Morph preserves on average 126% more functions, 60% more data types, and 73% more keywords than baselines. The diversity gap mainly comes from how each baseline handles unmappable constructs: statement-level translators raise errors or omit them, while Sedar comments them out, whereas Morph constructively instantiates abstract structures into target-compliant forms via retrieved dialect evidence, preserving more feature-rich inputs for mutation.

TABLE VI
Valid SQL features in transferred seeds.

Method	MonetDB			Virtuoso			DuckDB			ClickHouse		
	F	DT	K	F	DT	K	F	DT	K	F	DT	K
No translation	107	29	235	28	16	102	185	34	291	83	36	124
SQLGlot	-	-	-	-	-	-	195	28	288	103	35	142
jOOQ	-	-	-	-	-	-	187	22	253	94	34	104
CrackSQL	-	-	-	-	-	-	198	32	329	146	46	136
RISE	145	27	236	45	18	122	251	34	314	155	54	160
Sedar	132	29	244	52	17	136	214	39	340	227	61	152
Morph	161	29	370	87	24	335	456	63	476	332	64	187
vs. Sedar (%)	+22	+0	+52	+67	+41	+146	+113	+62	+40	+46	+5	+23

F/DT/K: functions/data types/keywords; “-”: unsupported dialects.

> Answer to RQ3: Morph outperforms baselines by 5%–62% in syntax correctness and 100%–450% in semantic correctness. It also preserves on average 126% more functions, 60% more data types, and 73% more keywords than baselines. The results demonstrate that the transfer methods in Morph produce seeds that are both executable and rich in testing-relevant logic.

E. RQ4: Contribution of Individual Components

To analyze the specific contributions of the individual components within Morph, we conducted an ablation study across two phases. For the translation phase, we evaluated four main variants using the task of PostgreSQL-to-DuckDB transfer: (1) Morph$_{\text{base}}$: the baseline using direct zero-shot LLM translation (i.e., prompting the LLM with the raw SQL and asking for a direct translation, unlike the No translation baseline which executes the raw, untranslated SQL directly); (2) Morph$_{\text{abs}}$: equipped solely with Structure-Preserving Abstraction; (3) Morph$_{\text{rag}}$: equipped solely with Dialect-Aware AQT Instantiation; (4) Morph$_{\text{full}}$: the complete framework integrating both components.

For the fuzzing phase, since the Feedback-Driven Adaptive Sanitizer operates independently during runtime fuzzing and does not affect translation quality, we further introduce Morph$_{direct}$ as a controlled variant where the runtime feedback loop is disabled and invalid seeds are directly discarded, and compare it against Morph$_{\text{full}}$ where the sanitizer actively repairs parsing errors to maximize seed acceptance.

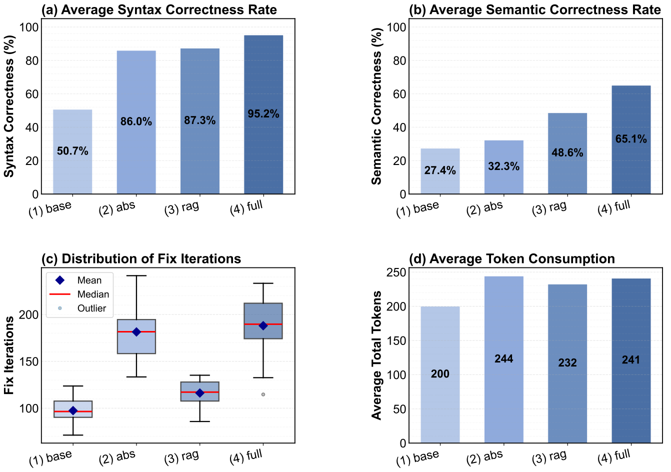


Fig. 8. Ablation Analysis of Morph Components. Comparison of syntactic/semantic correctness and resource consumption across four variants.

Contribution of Structure-Preserving Abstraction and Dialect-Aware AQT Instantiation. As shown in Figure ?? (a) and (b), when both the Structure-Preserving Abstraction and the Dialect-Aware AQT Instantiation are disabled, the syntactic correctness is 51% and the semantic correctness of Morph$_{\text{base}}$ is 27%. With Structure-Preserving Abstraction, Morph$_{\text{abs}}$ improves syntactic correctness to 86% (+35%) by leveraging AQT to provide a more stable structural skeleton. The semantic correctness also increases to 32%. Moreover, with Dialect-Aware AQT Instantiation, Morph$_{\text{rag}}$ further enhances syntactic correctness to 87%, and increases semantic correctness to 49% (+21%) by grounding generation in retrieved dialect evidence. The Full framework Morph$_{\text{full}}$ achieves

TABLE VII
Sanitizer ablation on branch coverage (24h).

DBMS	Morph$_{direct}$	Morph$_{\text{full}}$	Improvement
MonetDB	67,620	81,486	+21%
Virtuoso	37,213	40,183	+8%
DuckDB	118,453	144,822	+22%
ClickHouse	112,083	155,639	+39%

the optimal balance, reaching 95% syntactic and 65% semantic correctness.

Beyond dialect grounding, Dialect-Aware AQT Instantiation also propagates schema, type, and state context across statements to preserve test-case-level dependencies. To isolate this effect, we disable context propagation in the complete transfer pipeline while keeping statement-level instantiation and local dialect mappings unchanged. This reduces semantic correctness from 65.1% to 52.4% (-12.7%) while only slightly reducing syntactic correctness to 93.5% (-1.7%). The result shows that context propagation is a necessary part of Instantiation, mainly improving test-case-level executability. Figure ??(c) and (d) show that Morph$_{\text{full}}$ incurs more repair iterations and token overhead, but delivers the best trade-off between generation quality and efficiency.

Contribution of Feedback-Driven Adaptive Sanitizer. To evaluate the contribution of the component, we compared the full framework (Morph$_{\text{full}}$) against a variant named Morph$_{direct}$, where the runtime feedback loop is disabled. In the static variant, generated seeds are fed directly to the fuzzer without adaptive repair; if they fail parsing, they are simply discarded. Table ?? presents Morph covers 8% to 39% more branches across four DBMSs than Morph$_{direct}$ over a 24-hour campaign. When the component is disabled, Morph$_{direct}$ experiences a high rejection rate by the fuzzer at the parser stage due to syntax mismatches. In contrast, by actively repairing these specific errors, Morph$_{\text{full}}$ recovers these invalid seeds, allowing them to pass the frontend and be used for fuzzers to explore more state space of target DBMSs.

Answer to RQ4: All components are critical for Morph. Morph$_{\text{abs}}$ stabilizes the structural skeleton, improving syntax correctness by 35%. Morph$_{\text{rag}}$ ensures semantic alignment, boosting validity by 21%. Morph$_{\text{full}}$ achieves a improvement of 44% in syntax and 38% in semantics. Furthermore, the Feedback-Driven Sanitizer is indispensable for execution depth, boosting branch coverage by up to 39% (on ClickHouse) compared to static generation.

V. Discussion

LLM Parameter Selection. To ensure reproducibility, we conduct a sensitivity analysis on the temperature parameter (0.2, 0.5, 0.8). The results show that $T = 0.2$ achieves a 95% translation success rate and 91% first-attempt stability, outperforming $T = 0.5$ (92%/78%) and $T = 0.8$ (85%/70%). Based on these results, we set $T = 0.2$ for all experiments to ensure both reliability and reproducibility.

TABLE VIII
LLM comparison. TS denotes Test Suite.

Model	Syn.	Sem.	Cost/TS	Time/TS
Gemini 2.5-Flash-Lite	0.929	0.776	0.003	146.80
GPT-3.5-Turbo	0.890	0.533	0.015	208.66
DeepSeek-V3	0.986	0.699	0.003	231.11
Qwen3.5-27B	0.971	0.852	0.119	1395.6

* Qwen3.5-27B with deep thinking mode

TABLE IX
Manual effort for extending Morph to target DBMSs.

Target DBMS	Auto Rules	Manual	Ratio
DuckDB	534	42	7.3%
ClickHouse	1,362	46	3.3%
MonetDB	398	26	6.1%
Virtuoso	782	22	2.7%
Total	3,076	136	4.2%

Impact of the Underlying LLM. We justify the selection of Gemini 2.5-Flash-Lite for Morph using experiments on MonetDB with different LLMs. Although Qwen3.5-27B achieves slightly higher semantic correctness (0.852), it incurs 39.7× higher cost and 9.5× longer migration time. Gemini provides a better balance of accuracy (0.776), cost, and throughput, making it suitable for large-scale seed generation. Under the same GPT-3.5-Turbo setting as Sedar, Morph still outperforms it in syntactic (0.89 vs. 0.781) and semantic correctness (0.533 vs. 0.388), indicating that the gains mainly come from the proposed method rather than model capacity.

Extensibility and Manual Effort. To extend Morph to another DBMS, we first automatically extract lexical and syntactic features from grammar specifications and register the corresponding mapping rules into the Hybrid KB. For features that cannot be mapped automatically, Morph uses the LLM to generate candidate rules, followed by manual review and correction. Table ?? summarizes the extension effort across four target DBMSs. Overall, only 4.2% (136/3,212) of transfer rules require manual correction, and each correction typically takes 1–3 minutes for an engineer with DBMS expertise.

VI. Related Work

A. DBMS Fuzzing

Fuzzing is one of the most effective automated testing techniques for identifying DBMS vulnerabilities. Generation-based tools construct queries via predefined rules or semantic oracles. SQLsmith [?] randomly instantiates queries from a grammar. SQLancer [?], [?] applies oracles to detect logic bugs. DynSQL [?] leverages dynamic feedback to guide query mutation. More recently, clause-guided and differential methods have further improved logic-bug detection [?], [?], [?], [?], [?]. However, predefined templates can limit complex scenarios [?]. Mutation-based fuzzers generate test cases by continuously evolving a pool of seed inputs, rather than constructing them from scratch. Squirrel [?] uses IR-based mutations, Griffin [?] uses grammar-free metadata replacement, and LEGO [?] combines SQL statement sequences. These fuzzers are effective once suitable seeds or generation rules are available, but they do not directly address how to obtain high-quality seeds for emerging DBMSs.

Unlike these works, Morph focuses on the upstream seed acquisition problem by transferring mature DBMS test cases into target-compatible seed sequences.

B. LLM-Based Seed Generation

Recent work has explored LLMs for generating or enriching testing inputs. Fuzz4All [?] uses LLMs as a general input generator, and ShQveL [?] synthesizes SQL fragments for database testing. Sedar [?] is the closest work to Morph because it also transfers existing DBMS test cases into fuzzing seeds. However, these methods largely rely on direct text generation or post-generation filtering and sanitization.

In contrast, Morph performs AQT-guided whole-test-case instantiation, where explicit schema dependencies, nested structures, and clause-level relationships support local repair while preserving context.

C. Cross-Dialect SQL Translation

General-purpose SQL dialect translators, such as SQLGlot [?] and jOOQ [?], exploit the deterministic structure of SQL dialects and encode dialect mappings as parser or transpiler rules, making them useful for recurring statement-level conversions in query migration and application compatibility. However, constructing and maintaining these rules requires significant dialect expertise. Recent studies have reduced this manual effort by learning or deriving translation behaviors automatically. CrackSQL [?] employs genetic algorithms alongside LLMs to mine complex mutation patterns. QTRAN [?] extends metamorphic-oracle-based testing across DBMS dialects. RISE [?] translates a query by first reducing a complex input to a simpler form, learning the local dialect rewrite from that reduced form, and then applying the rewrite back to the original query. While this strategy improves robustness, it still treats a single query as the basic translation unit and does not account for multi-statement interactions.

Overall, existing SQL translation methods treat a single query as the translation unit rather than targeting fuzzing-oriented whole-test-case seed transfer where cross-statement dependencies, evolving schema context, and compatibility with downstream fuzzer grammars must be preserved. Unlike them, Morph transfers the entire seed sequence as normalized per-statement forms within an AQT while preserving cross-statement dependencies and evolving schema context.

VII. Conclusion

In this paper, we propose Morph to construct fuzzing seeds with structure-aware cross-dialect SQL transfer. It extracts dialect-agnostic AQTs, instantiates them through Dialect-Aware AQT Instantiation, and applies adaptive sanitization for cross-dialect DBMS fuzzing. Compared with the state-of-the-art approach Sedar, Morph covers 14% more branches and finds 16 more bugs in 24 hours. Across four popular DBMSs, Morph uncovers 28 previously unknown bugs, including 20 confirmed and 14 already fixed.

VIII. Data Availability

Artifacts and data are available on anonymous Zenodo [?].

References

[1] T. P. G. D. Group, “pgsql-bugs,” https://www.postgresql.org/list/pgsql-bugs/, 2025, accessed: October 23, 2026.

[2] MySQL, “Mysql bug home,” https://bugs.mysql.com/, 2025, accessed: October 23, 2026.

[3] M. Zalewski, “american fuzzy lop,” http://lcamtuf.coredump.cx/afl/, accessed: October 23, 2026.

[4] “Libfuzzer,” https://www.llvm.org/docs/LibFuzzer.html, accessed: October 23, 2026.

[5] A. Fioraldi, D. Maier, H. Eißfeldt, and M. Heuse, “Afl++ : Combining incremental steps of fuzzing research,” in USENIX Workshop on Offensive Technologies (WOOT), 2020.

[6] J. Liang, Y. Jiang, Y. Chen, M. Wang, C. Zhou, and J. Sun, “Pafl: Extend fuzzing optimizations of single mode to industrial parallel mode,” 2018.

[7] “Sqlsmith score list,” https://github.com/anse1/sqlsmith/wiki#score-list, accessed: October 23, 2026.

[8] “Bugs found in database management systems,” https://www.manuelrigger.at/dbms-bugs, accessed: October 23, 2026.

[9] SQLsmith Contributors, “Sqlsmith,” 2015, accessed: 2026-01-26. [Online]. Available: https://github.com/anse1/sqlsmith

[10] J. Fu, J. Liang, Z. Wu, M. Wang, and Y. Jiang, “Griffin: Grammar-free dbms fuzzing,” in Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering, 2022, pp. 1–12.

[11] J. Liang, Y. Chen, Z. Wu, J. Fu, M. Wang, Y. Jiang, X. Huang, T. Chen, J. Wang, and J. Li, “Sequence-oriented dbms fuzzing,” in 2023 IEEE 39th International Conference on Data Engineering (ICDE). IEEE, 2023, pp. 668–681.

[12] Z.-M. Jiang, J.-J. Bai, and Z. Su, “{DynSQL}: Stateful fuzzing for database management systems with complex and valid {SQL} query generation,” in 32nd USENIX Security Symposium (USENIX Security 23), 2023, pp. 4949–4965.

[13] R. Zhong, Y. Chen, H. Hu, H. Zhang, W. Lee, and D. Wu, “Squirrel: Testing database management systems with language validity and coverage feedback,” in Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security, 2020, pp. 955–970.

[14] “Postgresql github,” https://github.com/postgres/postgres, accessed: October 23, 2026.

[15] “Duckdb github,” https://github.com/duckdb/duckdb, accessed: October 23, 2026.

[16] X. Zhou, C. Chai, G. Li, and J. Sun, “Database meets artificial intelligence: A survey,” IEEE Transactions on Knowledge and Data Engineering, vol. 34, no. 3, pp. 1096–1116, 2020.

[17] C. S. Xia, M. Paltenghi, J. Le Tian, M. Pradel, and L. Zhang, "Fuzz4all: Universal fuzzing with large language models," in Proceedings of the IEEE/ACM 46th International Conference on Software Engineering, 2024, pp. 1–13.

[18] Y. Deng, C. S. Xia, H. Peng, C. Yang, and L. Zhang, "Large language models are zero-shot fuzzers: Fuzzing deep-learning libraries via large language models," in Proceedings of the 32nd ACM SIGSOFT international symposium on software testing and analysis, 2023, pp. 423–435.

[19] J. Fu, J. Liang, Z. Wu, and Y. Jiang, "Sedar: Obtaining high-quality seeds for dbms fuzzing via cross-dbms sql transfer," in Proceedings of the IEEE/ACM 46th International Conference on Software Engineering, 2024, pp. 1–12.

[20] SQLGlot Contributors, "SQLGlot: Python sql parser and transpiler," 2026, accessed: 2026-06-20. [Online]. Available: https://github.com/tobymao/sqlglot

[21] jOOQ Contributors, "jOOQ: Type safe sql in java," 2026, accessed: 2026-06-20. [Online]. Available: https://www.jooq.org/

[22] X. Xie, Y. Zhang, W. Dou, Y. Gao, Z. Cui, J. Song, R. Yang, and J. Wei, "Rise: Rule-driven sql dialect translation via query reduction," arXiv preprint arXiv:2601.05579, 2026.

[23] N. Mündler, J. He, H. Wang, K. Sen, D. Song, and M. Vechev, "Type-constrained code generation with language models," Proceedings of the ACM on Programming Languages, vol. 9, no. PLDI, pp. 601–626, 2025.

[24] L. Lin, Z. Hao, C. Wang, Z. Wang, R. Wu, and G. Fan, "Sqless: Dialect-agnostic sql query simplification," in Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis, 2024, pp. 743–754.

[25] "MonetDB NextRelease milestone," https://github.com/MonetDB/MonetDB/milestone/1?closed=1, 2025, accessed: 2026-03-25.

[26] M. Wang, Z. Wu, X. Xu, J. Liang, C. Zhou, H. Zhang, and Y. Jiang, "Industry practice of coverage-guided enterprise-level dbms fuzzing," in 2021 IEEE/ACM 43rd International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP). IEEE, 2021, pp. 328–337.

[27] C. Zhang, M. Bai, Y. Zheng, Y. Li, W. Ma, X. Xie, Y. Li, L. Sun, and Y. Liu, "Understanding large language model based fuzz driver generation," arXiv e-prints, pp. arXiv–2307, 2023.

[28] Google, "Gemini 2.5: Pushing the frontier with advanced reasoning, multimodality, long context, and next generation agentic capabilities," 2025. [Online]. Available: https://arxiv.org/abs/2507.06261

[29] "Db-engines ranking — popularity ranking of database management systems," https://db-engines.com/en/ranking, accessed: October 23, 2026.

[30] "The database system to speed up your analytical jobs," https://www.monetdb.org/, 1 2024, accessed: October 23, 2026.

[31] "Virtuoso github." https://github.com/openlink/virtuoso-opensource, accessed: October 23, 2026.

[32] "Query billions of rows in milliseconds," https://clickhouse.com/, 1 2024, accessed: October 23, 2026.

[33] S. Bhosale, M. T. Patil, and M. P. Patil, "Sqlite: Light database system," Int. J. Comput. Sci. Mob. Comput, vol. 44, no. 4, pp. 882–885, 2015.

[34] "Mysql," https://www.mysql.com/, 1 2024, accessed: October 23, 2026.

[35] "Postgresql," https://www.postgresql.org/, 1 2024, accessed: October 23, 2026.

[36] W. Zhou, Y. Gao, X. Zhou, and G. Li, "Cracking sql barriers: An llm-based dialect translation system," Proceedings of the ACM on Management of Data, vol. 3, no. 3, pp. 1–26, 2025.

[37] M. Rigger and Z. Su, "Sqlancer," 2023, accessed: 2026-01-26. [Online]. Available: https://github.com/sqlancer/sqlancer

[38] C. chen, "Squirrel website," https://github.com/s3team/Squirrel, accessed: October 23, 2026.

[39] M. Rigger and Z. Su, "Testing database engines via pivoted query synthesis," in 14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20), 2020, pp. 667–682.

[40] J. Wei, P. Chen, K. Lu, J. Dai, and X. Sun, "Sqlaser: Detecting dbms logic bugs with clause-guided fuzzing," arXiv preprint arXiv:2407.04294, 2024.

[41] J. Ba and M. Rigger, "Testing database engines via query plan guidance," in 2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE). IEEE, 2023, pp. 2060–2071.

[42] J. Song, W. Dou, Z. Cui, Q. Dai, W. Wang, J. Wei, H. Zhong, and T. Huang, "Testing database systems via differential query execution," in 2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE). IEEE, 2023, pp. 2072–2084.

[43] Z. Cui, W. Dou, Q. Dai, J. Song, W. Wang, J. Wei, and D. Ye, "Differentially testing database transactions for fun and profit," in Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering, 2022, pp. 1–12.

[44] Z. Hao, Q. Huang, C. Wang, J. Wang, Y. Zhang, R. Wu, and C. Zhang, "Pinolo: Detecting logical bugs in database management systems with approximate query synthesis," in 2023 USENIX Annual Technical Conference (USENIX ATC 23), 2023, pp. 345–358.

[45] X. Gao, Z. Liu, J. Cui, H. Li, H. Zhang, K. Wei, and K. Zhao, "A comprehensive survey on database management system fuzzing: Techniques, taxonomy and experimental comparison," arXiv preprint arXiv:2311.06728, 2023.

[46] S. Zhong and M. Rigger, "Testing database systems with large language model synthesized fragments," arXiv preprint arXiv:2505.02012, 2025.

[47] L. Lin, Q. Zhu, H. Chen, Z. Wang, R. Wu, and X. Xie, "Qtran: Extending metamorphic-oracle based logical bug detection techniques for multiple-dbms dialect support," Proceedings of the ACM on Software Engineering, vol. 2, no. ISSTA, pp. 731–752, 2025.

[48] "Morph artifact and experimental data," Anonymous Zenodo archive, 2026. [Online]. Available: https://doi.org/10.5281/zenodo.19251660